校园二手交易平台 - 系统架构文档

文档信息

- **项目名称**: 校园二手交易平台(SE2_NJU_Market)
- **文档版本**: v1.0
- **创建日期**: 2026-05-01
- **最后更新**: 2026-05-01
- **文档作者**: 全体团队成员
- **文档状态**: 正式发布

目录

1. 系统概述
2. 架构设计
3. 模块设计
4. 技术架构
5. 数据架构
6. 接口设计
7. 部署架构
8. 安全架构
9. 性能考虑
10. 质量属性
11. 演进路线

---

1. 系统概述

1.1 项目背景

校园二手交易平台是一个面向校内学生的二手商品交易系统,旨在为学生提供一个安全、便捷的二手商品发布和交易平台。通过该平台,学生可以发布闲置物品、浏览搜索商品、在线沟通、完成交易,并进行评价。

1.2 系统目标

1. **核心功能**: 提供完整的二手商品交易流程
2. **用户体验**: 简洁易用的界面,流畅的操作体验
3. **安全可靠**: 保证交易安全,用户信息安全
4. **可维护性**: 代码结构清晰,便于后期维护和扩展
5. **可演示性**: 10周内完成可运行原型

1.3 项目约束

- **开发周期**: 10周
- **团队规模**: 4人
- **技术背景**: 团队成员对Java和JavaScript有基础
- **项目性质**: 软件工程二课程项目

1.4 核心功能

功能模块	主要功能
用户管理	注册、登录、个人信息管理、头像上传
商品管理	发布商品、编辑商品、删除商品、商品图片管理
商品浏览	商品列表、商品搜索、商品详情、分类浏览
订单管理	创建订单、订单列表、订单详情、订单状态管理
评价系统	商品评价、评价列表、评分统计

功能模块	主要功能
消息系统	商品评论、评论回复
收藏功能	收藏商品、收藏列表
举报管理	举报商品/用户、举报处理

---

2. 架构设计

2.1 架构风格

本项目采用**前后端分离 + 分层架构 + MVC模式的模块化单体架构。**

详细决策理由参见 [ADR-001: 架构风格选择](#)。

2.2 架构层次

```
┌──────────────────────────────────────────────────────────────┐
│                          表现层                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 前端应用 (Vue 3)                                      │    │
│  │ - 页面组件                                            │    │
│  │ - 路由管理                                            │    │
│  │ - 状态管理                                            │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                            │ HTTP/REST API
                            ↓
┌──────────────────────────────────────────────────────────────┐
│                      应用层(Controller)                      │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Spring Boot Controllers                               │    │
│  │ - 接收HTTP请求                                        │    │
│  │ - 参数校验                                            │    │
│  │ - 调用业务逻辑                                        │    │
│  │ - 返回HTTP响应                                        │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                            │
                            ↓
┌──────────────────────────────────────────────────────────────┐
│                    业务逻辑层(Service)                       │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Spring Boot Services                                  │    │
│  │ - 业务逻辑处理                                        │    │
│  │ - 事务管理                                            │    │
│  │ - 业务规则验证                                        │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                            │
                            ↓
┌──────────────────────────────────────────────────────────────┐
│                  数据访问层(Repository)                      │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ MyBatis Plus Mappers                                  │    │
│  │ - 数据库访问                                          │    │
│  │ - SQL执行                                             │    │
│  │ - ORM映射                                             │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                            │
                            ↓
┌──────────────────────────────────────────────────────────────┐
│                          数据层                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ MySQL 数据库                                          │    │
│  │ - 数据持久化                                          │    │
│  │ - 事务支持                                            │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

2.3 架构特点

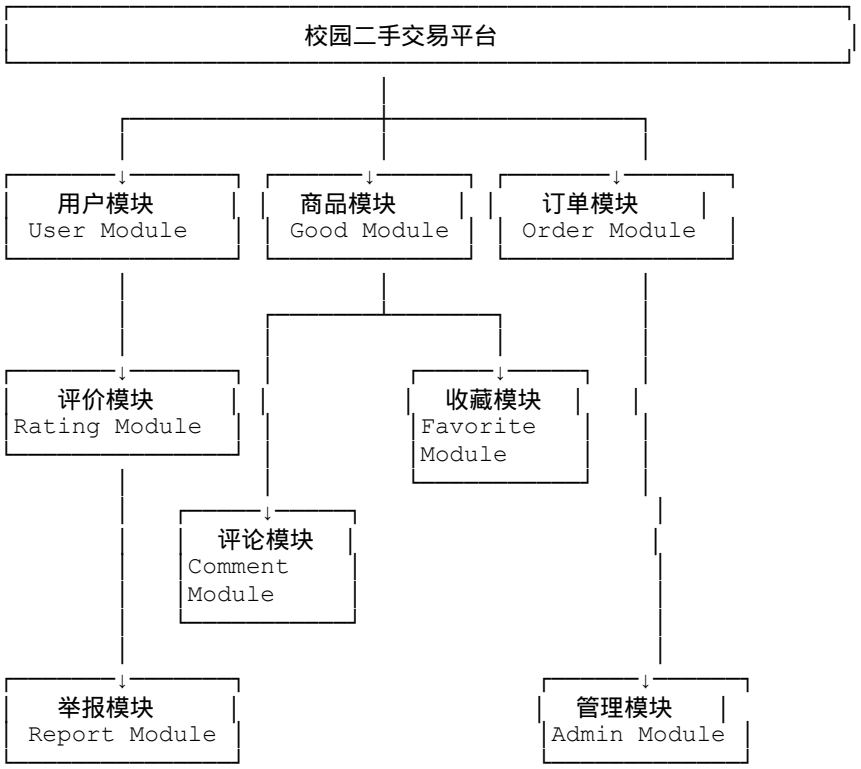
1. **前后端分离**: 前端和后端独立开发、独立部署,通过RESTful API通信
2. **分层清晰**: Controller、Service、Repository职责明确,降低耦合
3. **模块化**: 后端按业务模块划分,边界清晰
4. **单体架构**: 所有模块部署在一个应用中,降低运维复杂度
5. **可演进**: 模块化设计为未来拆分服务保留空间

---

3. 模块设计

3.1 模块划分

系统按业务功能划分为以下模块:



3.2 模块职责

用户模块 (User Module)

职责: 用户认证、授权、个人信息管理

核心功能:

- 用户注册
- 用户登录/登出
- 个人信息查看与修改
- 头像上传
- 密码修改

核心类:

- `UserController`: 用户接口控制器
- `UserService`: 用户业务逻辑
- `UserMapper`: 用户数据访问
- `User`: 用户实体类

商品模块 (Good Module)

职责: 商品发布、管理、浏览、搜索

核心功能:

- 商品发布
- 商品编辑
- 商品删除
- 商品列表查询
- 商品搜索
- 商品详情查看
- 商品图片管理
- 商品分类浏览

核心类:

- `GoodController`: 商品接口控制器
- `GoodService`: 商品业务逻辑
- `GoodMapper`: 商品数据访问
- `Good`: 商品实体类
- `GoodImageController`: 商品图片控制器
- `GoodImageService`: 商品图片业务逻辑
- `GoodImageMapper`: 商品图片数据访问
- `GoodImage`: 商品图片实体类

订单模块 (Order Module)

职责: 订单创建、管理、查询

核心功能:

- 创建订单
- 订单列表查询
- 订单详情查看
- 订单状态更新
- 订单项管理

核心类:

- `OrderEntityController`: 订单接口控制器
- `OrderEntityService`: 订单业务逻辑
- `OrderEntityMapper`: 订单数据访问
- `OrderEntity`: 订单实体类
- `OrderItemController`: 订单项控制器
- `OrderItemService`: 订单项业务逻辑
- `OrderItemMapper`: 订单项数据访问
- `OrderItem`: 订单项实体类

评价模块 (Rating Module)

职责: 商品评价、评分

核心功能:

- 提交评价
- 评价列表查询
- 评分统计

核心类:

- `RatingController`: 评价接口控制器
- `RatingService`: 评价业务逻辑

- `RatingMapper`: 评价数据访问
- `Rating`: 评价实体类

评论模块 (Comment Module)

职责: 商品评论、回复

核心功能:

- 发表评论
- 评论回复
- 评论列表查询
- 删除评论

核心类:

- `CommentController`: 评论接口控制器
- `CommentService`: 评论业务逻辑
- `CommentMapper`: 评论数据访问
- `Comment`: 评论实体类

收藏模块 (Favorite Module)

职责: 商品收藏

核心功能:

- 收藏商品
- 取消收藏
- 收藏列表查询

核心类:

- `FavoriteController`: 收藏接口控制器
- `FavoriteService`: 收藏业务逻辑
- `FavoriteMapper`: 收藏数据访问
- `Favorite`: 收藏实体类

举报模块 (Report Module)

职责: 举报管理

核心功能:

- 举报商品/用户
- 举报列表查询
- 举报处理

核心类:

- `ReportController`: 举报接口控制器
- `ReportService`: 举报业务逻辑
- `ReportMapper`: 举报数据访问
- `Report`: 举报实体类

管理模块 (Admin Module)

职责: 系统管理、举报处理

核心功能:

- 管理员登录
- 用户管理
- 商品管理
- 订单管理
- 举报处理

核心类:

- `AdminController`: 管理接口控制器
- `AdminService`: 管理业务逻辑
- `AdminMapper`: 管理数据访问
- `Admin`: 管理员实体类

3.3 模块依赖关系

```
用户模块 ←──────┐
    │           │
    ↓           │
商品模块 ←──────┤
    │           │
    ↓           │
订单模块        │
    │           │
    ↓           │
评价模块 ───────┤
    │           │
    ↓           │
评论模块        │
    │           │
    ↓           │
收藏模块 ───────┤
    │           │
    ↓           │
举报模块 ───────┘
    │
    ↓
管理模块
```

依赖说明:

- 所有模块都依赖用户模块(需要用户信息)
- 订单、评价、评论、收藏模块依赖商品模块
- 举报模块依赖用户模块和商品模块
- 管理模块依赖所有其他模块

---

4. 技术架构

4.1 技术栈

详细技术选型参见 [ADR-002: 技术栈选择]。

后端技术栈

层次	技术	版本	说明
框架	Spring Boot	3.3.5	后端框架
语言	Java	17	编程语言
ORM	MyBatis Plus	3.5.7	ORM框架
数据库	MySQL	8.0.32	关系型数据库
工具	Lombok	1.18.26	代码简化
文档	Swagger	1.5.1	API文档

前端技术栈

层次	技术	版本	说明
框架	Vue.js	3.5.32	前端框架
构建工具	Vite	8.0.4	构建工具
语言	JavaScript ES6+		编程语言

4.2 技术架构图

```
┌──────────────────────────────────────────────────────────────┐
│                          前端技术栈                          │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Vue 3.5.32                                           │    │
│  │ - 组合式API                                          │    │
│  │ - 响应式系统                                         │    │
│  │ - 组件化开发                                         │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Vite 8.0.4                                           │    │
│  │ - 开发服务器                                         │    │
│  │ - 热模块替换                                         │    │
│  │ - 生产构建                                           │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ HTTP/REST
                              ↓
┌──────────────────────────────────────────────────────────────┐
│                          后端技术栈                          │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Spring Boot 3.3.5                                    │    │
│  │ - Spring MVC (Web层)                                 │    │
│  │ - Spring Core (IoC/AOP)                              │    │
│  │ - Spring Transaction (事务管理)                      │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ MyBatis Plus 3.5.7                                   │    │
│  │ - ORM映射                                            │    │
│  │ - CRUD接口                                           │    │
│  │ - 代码生成器                                         │    │
│  │ - 分页插件                                           │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Lombok 1.18.26                                       │    │
│  │ - 自动生成getter/setter                              │    │
│  │ - 自动生成构造函数                                   │    │
│  │ - 日志注解                                           │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Swagger 1.5.1                                        │    │
│  │ - API文档生成                                        │    │
│  │ - 接口测试                                           │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ JDBC
                              ↓
┌──────────────────────────────────────────────────────────────┐
│                        数据存储层                            │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ MySQL 8.0.32                                         │    │
│  │ - InnoDB存储引擎                                     │    │
│  │ - 事务支持                                           │    │
│  │ - 外键约束                                           │    │
│  │ - 索引优化                                           │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

4.3 开发工具

工具类型	工具名称	用途
IDE	IntelliJ IDEA	后端开发
IDE	VS Code	前端开发
版本控制	Git	代码版本管理
项目管理	Maven	后端项目构建

工具类型	工具名称	用途
包管理	npm	前端包管理
数据库工具	MySQL Workbench	数据库管理

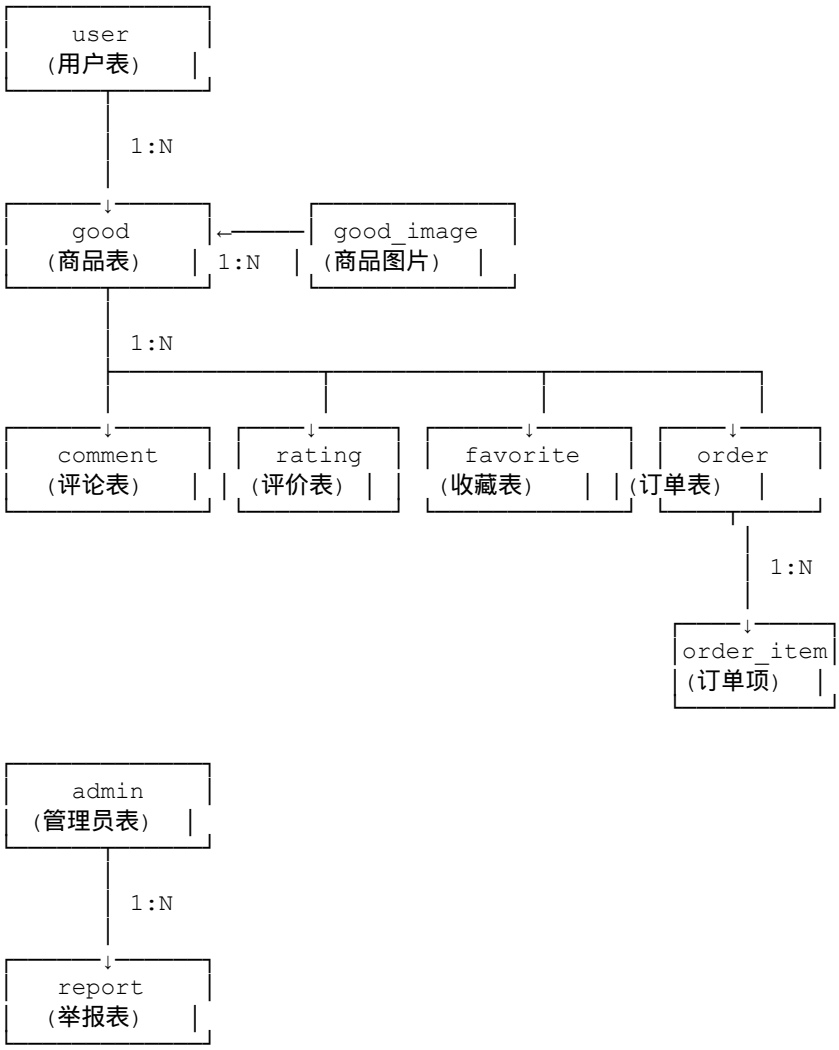
---

5. 数据架构

5.1 数据库设计

详细数据库设计参见 [ADR-003: 数据库设计决策](#)。

5.2 ER图



5.3 核心表结构

用户表 (user)

字段名	类型	说明
user_id	INT	主键,自增
username	VARCHAR(100)	用户名
password	VARCHAR(255)	密码(加密)
email	VARCHAR(120)	邮箱
phone	VARCHAR(30)	手机号
avatar_url	VARCHAR(255)	头像URL
campus	VARCHAR(100)	校区
address	VARCHAR(255)	地址
created_at	TIMESTAMP	创建时间

字段名	类型	说明
updated_at	TIMESTAMP	更新时间

商品表 (good)

字段名	类型	说明
good_id	INT	主键,自增
seller_id	INT	卖家ID(外键)
title	VARCHAR(150)	商品标题
description	TEXT	商品描述
price	DECIMAL(10,2)	价格
category	VARCHAR(100)	分类
status	VARCHAR(50)	状态
condition	VARCHAR(50)	成色
view_count	INT	浏览次数
created_at	TIMESTAMP	创建时间
updated_at	TIMESTAMP	更新时间

订单表 (order)

字段名	类型	说明
order_id	INT	主键,自增
buyer_id	INT	买家ID(外键)
seller_id	INT	卖家ID(外键)
order_number	VARCHAR(100)	订单号(唯一)
total_amount	DECIMAL(10,2)	总金额
status	VARCHAR(50)	订单状态
created_at	TIMESTAMP	创建时间
updated_at	TIMESTAMP	更新时间

5.4 数据字典

完整的数据字典请参见数据库设计文档。

---

6. 接口设计

6.1 API设计原则

1. **RESTful风格**: 使用HTTP动词表示操作(GET查询, POST创建, PUT更新, DELETE删除)
2. **统一响应格式**: 所有接口返回统一的JSON格式
3. **版本控制**: 通过URL路径进行版本控制(如/api/v1/)
4. **错误处理**: 统一的错误码和错误信息

6.2 统一响应格式

```
{
  "code": 200,
  "message": "success",
  "data": {
  }
}
```

6.3 核心API列表

用户模块API

方法	路径	说明
POST	/api/users/register	用户注册
POST	/api/users/login	用户登录
GET	/api/users/{id}	获取用户信息
PUT	/api/users/{id}	更新用户信息
POST	/api/users/avatar	上传头像

商品模块API

方法	路径	说明
POST	/api/goods	发布商品
GET	/api/goods	商品列表
GET	/api/goods/{id}	商品详情
PUT	/api/goods/{id}	更新商品
DELETE	/api/goods/{id}	删除商品
GET	/api/goods/search	搜索商品

订单模块API

方法	路径	说明
POST	/api/orders	创建订单
GET	/api/orders	订单列表
GET	/api/orders/{id}	订单详情
PUT	/api/orders/{id}/status	更新订单状态

6.4 API文档

API详细文档通过Swagger自动生成,访问地址: `http://localhost:8080/swagger-ui.html`

---

7. 部署架构

7.1 部署方案

本项目采用传统的三层部署架构:

```
┌──────────────────────────────────────────────────────────────┐
│                          客户端层                            │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 浏览器 (Chrome/Firefox/Safari/Edge)                  │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ HTTPS
                              ↓
┌──────────────────────────────────────────────────────────────┐
│                          应用层                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Web服务器 (Nginx/Apache)                             │    │
│  │ - 静态资源服务                                       │    │
│  │ - 反向代理                                           │    │
│  │ - 负载均衡                                           │    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 应用服务器 (Tomcat embedded in Spring Boot)          │    │
│  │ - 运行Spring Boot应用                                │    │
│  │ - 处理业务请求                                       │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ JDBC
                              ↓
┌──────────────────────────────────────────────────────────────┐
```

```
│                    数据层                                  │
│  ┌──────────────────────────────────────────────────┐    │
│  │  MySQL数据库                                      │    │
│  │  - 数据持久化                                     │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

7.2 部署环境

开发环境

- **前端**: Vite开发服务器 (localhost:5173)
- **后端**: Spring Boot内嵌Tomcat (localhost:8080)
- **数据库**: MySQL本地实例 (localhost:3306)

生产环境

- **前端**: Nginx静态资源服务器
- **后端**: 独立服务器运行Spring Boot JAR包
- **数据库**: MySQL独立服务器

7.3 部署步骤

前端部署

```
cd frontend
npm install
npm run build
```

将生成的`dist`目录部署到Nginx。

后端部署

```
cd backend
mvn clean package
java -jar target/boy-0.0.1-SNAPSHOT.jar
```

---

8. 安全架构

8.1 安全威胁分析

威胁类型	描述	风险等级
SQL注入	通过输入恶意SQL语句攻击数据库	高
XSS攻击	通过注入恶意脚本攻击用户浏览器	中
CSRF攻击	伪造用户请求攻击	中
密码泄露	用户密码被窃取	高
数据泄露	敏感数据被非法访问	高

8.2 安全措施

8.2.1 认证与授权

- **用户认证**: 用户名+密码登录
- **密码加密**: 使用BCrypt加密存储密码
- **会话管理**: 使用JWT Token管理用户会话
- **权限控制**: 基于角色的访问控制(RBAC)

8.2.2 数据安全

- **SQL注入防护**: 使用MyBatis Plus参数化查询
- **XSS防护**: 前端输入验证和转义

- **CSRF防护**: 使用CSRF Token
- **HTTPS**: 生产环境使用HTTPS加密传输
- **数据加密**: 敏感数据加密存储

8.2.3 接口安全

- **接口鉴权**: 所有接口需要验证Token
- **参数验证**: 后端参数校验
- **频率限制**: 接口访问频率限制
- **日志记录**: 记录关键操作日志

---

9. 性能考虑

9.1 性能目标

指标	目标值
页面加载时间	< 2秒
API响应时间	< 500ms
并发用户数	100
数据库查询时间	< 100ms

9.2 性能优化策略

9.2.1 前端优化

1. **代码分割**: 使用Vite进行代码分割,按需加载
2. **资源压缩**: 压缩JS、CSS、图片等资源
3. **懒加载**: 图片懒加载,路由懒加载
4. **缓存策略**: 合理使用浏览器缓存

9.2.2 后端优化

1. **数据库优化**:
    - 为常用查询字段建立索引
    - 使用MyBatis Plus分页插件
    - 避免N+1查询问题
2. **代码优化**:
    - 使用连接池管理数据库连接
    - 合理使用缓存
    - 异步处理耗时操作

9.2.3 数据库优化

1. **索引优化**: 为常用查询字段建立索引
2. **查询优化**: 优化慢查询SQL
3. **连接池**: 使用HikariCP连接池
4. **读写分离**: 未来可考虑读写分离

---

10. 质量属性

10.1 可维护性

目标: 代码结构清晰,易于理解和修改

措施:

1. 遵循分层架构,职责明确
2. 遵循代码规范,统一编码风格
3. 编写清晰的注释和文档
4. 定期代码审查

10.2 可扩展性

目标: 系统易于扩展新功能

措施:

1. 模块化设计,模块边界清晰
2. 面向接口编程,降低耦合
3. 使用设计模式,提高代码复用性
4. 预留扩展点

10.3 可测试性

目标: 系统易于测试

措施:

1. 单元测试: 为核心业务逻辑编写单元测试
2. 集成测试: 测试模块间集成
3. 接口测试: 使用Swagger测试API
4. 测试数据: 准备充分的测试数据

10.4 可部署性

目标: 系统易于部署

措施:

1. 使用Maven和npm自动化构建
2. 编写详细的部署文档
3. 使用Docker容器化部署(可选)
4. 自动化部署脚本

---

11. 演进路线

11.1 短期演进(0-6个月)

目标: 完善核心功能,优化用户体验

计划:

1. 完善商品搜索功能(全文搜索、筛选)
2. 增加用户消息通知功能
3. 优化前端界面和交互
4. 性能优化和bug修复

11.2 中期演进(6-12个月)

目标: 提升系统性能和可用性

计划:

1. 引入Redis缓存,提升性能
2. 数据库读写分离
3. 增加推荐算法
4. 移动端适配或开发小程序

11.3 长期演进(12个月+)

目标: 支撑更大规模用户和业务

计划:

1. 将消息通知、搜索推荐等模块拆分为独立服务
2. 引入消息队列,实现异步处理
3. 微服务化改造(如果业务规模需要)
4. 分布式部署和负载均衡

附录

A. 架构决策记录

- ADR-001: 架构风格选择
- ADR-002: 技术栈选择
- ADR-003: 数据库设计决策

B. 相关文档

- 架构候选方案对比表
- 架构辩论赛记录
- 需求规格说明书

C. 参考资料

- "Software Architecture in Practice" by Len Bass et al.
- "Building Evolutionary Architectures" by Neal Ford et al.
- Spring Boot官方文档: https://spring.io/projects/spring-boot
- Vue.js官方文档: https://vuejs.org/
- MyBatis Plus官方文档: https://baomidou.com/

文档结束